Project: Data Structures in Java
&
Time Complexity

COMFCI25/ Askerova Diana / 250142028

Option A — Cafeteria Line Manager (Queue with ArrayDeque

Research Brief:

Concepts: Big-O/Ω/Θ, worst vs. average, amortized analysis:

Big-O Notation

Big-O notation describes the **upper bound** of an algorithm's running time. It shows the maximum amount of time an algorithm may take as the input size grows. In other words, it represents the worst possible growth rate of the algorithm.

For example, an algorithm that scans through an array of size *n* once has time complexity **O(n)** because the number of operations increases linearly with the input size.

Big-O notation simplifies expressions by ignoring constants and lower-order terms. For example, **O(5n)** becomes **O(n)** and **O(n^2 + n)** becomes **O(n^2)** because the dominant term determines the growth rate.

Big-O is widely used in algorithm analysis because it helps predict performance when the dataset becomes very large.

Big-Ω (Omega) Notation

Big-Ω notation represents the **lower bound** of an algorithm's running time. It describes the best possible performance of the algorithm.

For example, searching an element in an unsorted array may take **Ω(1)** time if the element is located at the first position. This means the algorithm cannot perform faster than this lower bound.

Omega notation helps understand the minimum amount of work required by an algorithm.

Big-Θ (Theta) Notation

Big-Θ notation represents a **tight bound** of algorithm complexity. This means the algorithm's running time grows at the same rate in both upper and lower bounds.

If an algorithm has complexity **Θ(n)**, it means that the running time grows linearly with the input size and both upper and lower bounds are proportional to *n*.

Big-Theta is useful when the exact growth rate of an algorithm is known.

A short table of **operation complexities**(insert/search/delete/access) for arrays, ArrayList, HashSet, HashMap, ArrayDeque

Data structue	Access	Search	Insert	Delete
Array	O(1)	O(n)	O(n)	O(n)
ArreyList	O(1)	O(n)	O(n)amortizet	O(n)
HashSet	--	O(1)average	O(1)	O(1)
HashMap	--	O(1)average	O(1)	O(1)
ArrayDeque	O(1)	O(n)	O(1)	O(1)

Arrays allow constant-time access because elements are stored in contiguous memory and can be accessed directly using their index. However, inserting or deleting elements may require shifting other elements, which takes linear time.

ArrayList is a dynamic array that automatically resizes when its capacity is exceeded. Most insert operations at the end are constant time, but occasionally resizing occurs, resulting in amortized complexity.

Hash-based structures such as HashSet and HashMap provide very fast average-case operations because they use hashing to directly locate elements.

ArrayDeque is typically used for queues and stacks, allowing constant-time insertion and removal from both ends.

When to choose which structure for practical scenario

In the Cafeteria Line Manager simulation, the main task is to represent a line of people waiting to be served. The most appropriate data structure for this situation is a **queue**, because a queue follows the **First-In-First-Out (FIFO)** principle. This means that the first person who joins the line will be the first person to be served, which reflects how real cafeteria lines work. In Java, the **ArrayDeque** class is a good implementation of a queue because it allows fast insertion at the end of the line and removal from the front, both with **O(1)** time complexity.

The program also needs to store the arrival time of each person in order to calculate how long they waited before being served. For this purpose, a **HashMap** is an effective choice. A HashMap allows the program to associate each person's name with their arrival time, enabling quick access to that information when the person is served. Since HashMap operations such as insertion and lookup typically run in **O(1)** average time, the program can efficiently track waiting times even when many people are in the system.

Combining these data structures makes the simulation efficient and realistic. The **ArrayDeque** manages the order of people in the line, while the **HashMap** keeps track of arrival times needed to compute waiting statistics. This separation of responsibilities keeps the program organized and ensures that both queue operations and data lookups remain fast as the number of customers increases.

References

1. Cormen, T., Leiserson, C., Rivest, R., Stein, C. *Introduction to Algorithms*. MIT Press.
2. Joel's Dev Blog. "Data Structures and Algorithms Cheatsheet." https://9oelm.github.io/2022-09-15-data-structures-and-algorithms-cheatsheet/
3. Wikipedia. "Amortized Analysis." https://en.wikipedia.org/wiki/Amortized_analysis

4. Sarthaks. “Data Structures Asymptotic Analysis.” https://www.sarthaks.com/3565230/data-structures-asymptotic-analysis

Short Report

Cafeteria Line Manager – Data Structures and Testing

1. Data Structure Choices and Time Complexity

The Cafeteria Line Manager simulation models a real-world waiting line in a cafeteria. The main goal is to manage people arriving, waiting, and being served while tracking waiting times. To implement this system efficiently, two main data structures were used: **ArrayDeque** and **HashMap**.

The queue of customers is implemented using **ArrayDeque**. This structure is ideal because it efficiently supports operations at both ends of the queue. In the simulation, new customers join the back of the line using addLast(), while VIP customers are inserted at the front using addFirst(). When a person is served, they are removed from the front using removeFirst(). All these operations have **amortized time complexity O(1)**, which means they execute very quickly even when the queue becomes large.

A **HashMap<String, Integer>** is used to store the arrival time of each person. The key is the person’s name, and the value is the logical time when they joined the line. This allows the program to quickly retrieve the arrival time when the person is served in order to compute their waiting time. HashMap operations such as put() and get() have **average time complexity O(1)**, making them efficient for frequent lookups.

Some operations require scanning the queue. For example, the LEAVE <name> command uses removeFirstOccurrence(name) to remove a person from the line. Since the queue must be searched sequentially, this operation has **O(n)** time

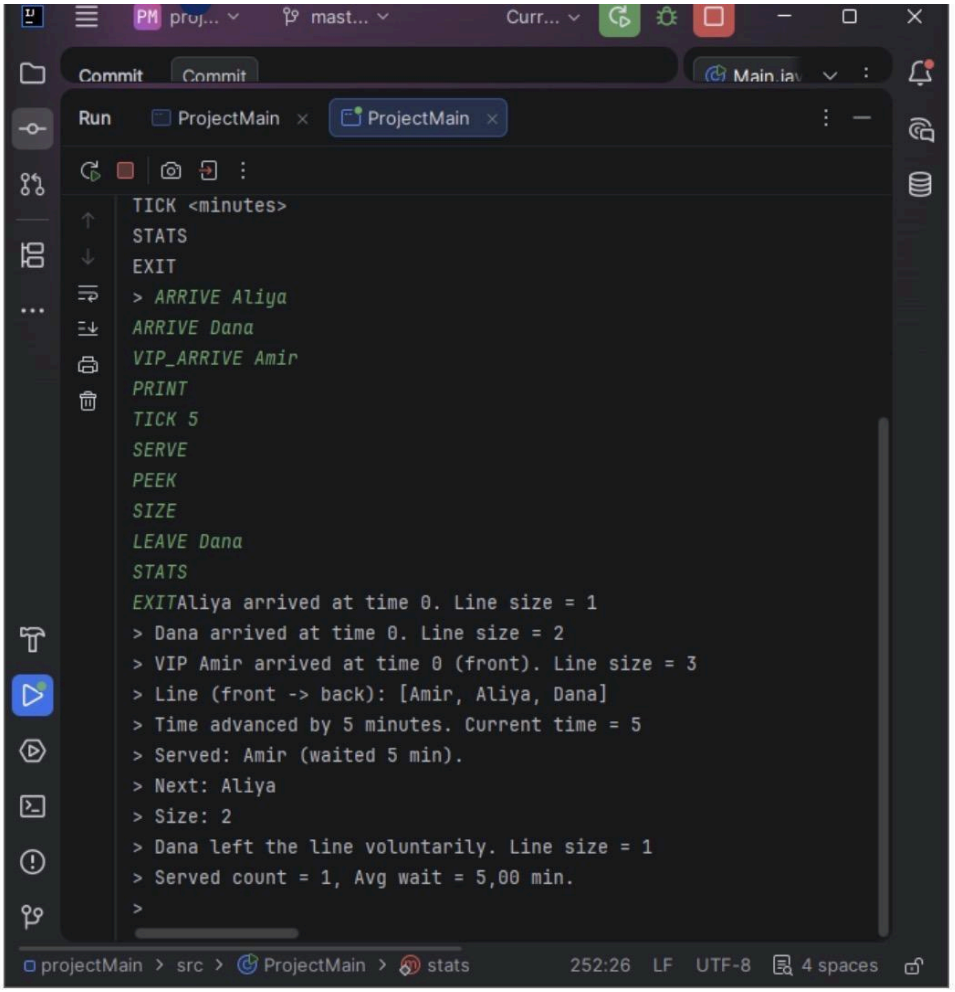
complexity. However, this operation is expected to occur less frequently than basic queue operations, so the overall system remains efficient.

2. Manual Test Log

The program was manually tested using several command sequences to verify that all operations behave correctly.

Example test session:

Input- output



The test confirmed that:

• VIP customers correctly move to the front of the queue.

• Waiting time is calculated using the logical clock.
• Queue size updates correctly after arrivals, serving, and leaving.
• Statistics correctly track served customers and average wait time.

3. Small Empirical Timing Observation

To observe the performance difference between operations, a small experiment was conducted using System.nanoTime().

The goal was to compare the execution time of two operations:

1. Adding an element to the queue (addLast)
2. Removing an element by name (removeFirstOccurrence)

Observation

• The addLast() operation executes in constant time regardless of the queue size.

• The removeFirstOccurrence() operation takes longer when the queue size increases because the algorithm must search through the queue to find the element.

This experiment confirms the theoretical complexity expectations: queue insertion operations run in O(1) time, while searching and removing by value requires O(n) time.